

COMS 4130 Detailed Design Document

<https://github.com/shashankkoneru/tcas>

Team Members:

Ryan Horsey, Kaitlyn Hoyme, Arnold Joy, Shashank Koneru, Kennedy Wendl

Static Analysis Component

The static analysis part of our project focuses on understanding the structure and decision logic of the TCAS program without actually running it. Since our goal is to build a system that can answer questions about program analysis results, we first needed to extract useful structural information from the code and store it in a form that can be queried later.

We used `tcas.c` as the target for static analysis. It is a good fit because it is small enough to manage, but still has enough branching and function interaction to make the analysis meaningful. In particular, the main advisory logic in `alt_sep_test()` depends on several helper functions and multiple conditional checks, which makes it useful for call graph, control flow, and dependency analysis.

Our main static analysis tasks were generating a call graph, control flow graphs (CFGs), an interprocedural control flow graph (ICFG), and a dependency summary.

Call Graph and CFG Generation

We used LLVM to generate a call graph showing how functions in `tcas` relate to each other. This helped confirm that `main` is mainly responsible for reading inputs and calling `alt_sep_test()`, while `alt_sep_test()` contains the main decision logic and calls helper functions such as `Non_Crossing_Biased_Climb()`, `Non_Crossing_Biased_Descend()`, `Own_Below_Threat()`, and `Own_Above_Threat()`.

We also generated CFGs for the program's major functions. These were produced as `.dot` files using LLVM and then converted into `.png` images with Graphviz. The most important CFG was for `alt_sep_test()`, since it shows how the program checks whether the advisory logic is enabled, determines whether upward or downward advisories are needed, and then returns one of three outcomes: `UNRESOLVED`, `UPWARD_RA`, or `DOWNWARD_RA`. The CFGs for `Non_Crossing_Biased_Climb()` and `Non_Crossing_Biased_Descend()` were also useful because they showed the nested branch logic used to evaluate climb and descend conditions.

Interprocedural Control Flow Graph

In addition to per-function CFGs, we created an ICFG for TCAS. While a normal CFG shows control flow within a single function, the ICFG connects control flow across function boundaries. This gave us a more complete picture of how execution moves through the full program.

The ICFG was especially useful because TCAS relies on several helper functions in its advisory logic. By combining control flow with function-call relationships, the ICFG provides a clearer overall view of how the program operates.

Dependency Analysis

We also created a dependency summary to describe how the program's variables and decision points depend on one another. This helped us understand which values directly affect the final advisory result.

We also traced dependencies within helper functions such as `Non_Crossing_Biased_Climb()` and `Non_Crossing_Biased_Descend()`, which depend on values such as `Up_Separation`, `Down_Separation`, `Cur_Vertical_Sep`, `Alt_Layer_Value`, and the result of `ALIM()`.

Tools Used and Project Role

The main tool used for static analysis was LLVM. We used LLVM bitcode and the `opt` tool to generate the call graph and CFGs. We also used Graphviz to convert `.dot` files into `.png` images for easier viewing. The static analysis component is the foundation for the rest of the project, since the query system will later depend on these stored structural results to answer questions about the logic and organization of TCAS.

Testing Component

The testing component of our project focuses on how TCAS behaves when it is actually executed. Since the static analysis side tells us how the program is structured, the testing side is meant to show what happens under real, mutated, and large benchmark test inputs. Our testing design was built around three sources: the pre-existing SIR benchmark test suite, a smaller hand-crafted test suite, and fuzz testing with AFL++. We also used gcov to collect statement and branch coverage.

Overall Testing Strategy

We did not want to rely on only one type of testing. The benchmark suite produced a large number of executions, the hand-crafted suite provided smaller, verified examples that were easier to understand, and AFL++ enabled us to explore abnormal inputs and potential failure cases.

Pre-Existing and Hand-Crafted Test Suites

Our main baseline testing source was the universe file from the SIR benchmark distribution, which contains 1,608 TCAS test scenarios. We wrote a script, `run_universe.sh`, to run these inputs and save the outputs. Some of the final rows in the file were intentionally malformed and triggered the program's usage-error path, so we separated valid twelve-argument rows from malformed ones for more accurate coverage and structured logging.

We also created our own hand-crafted test suite at `testing/test_cases.txt`. The purpose of this smaller suite was to give us a set of verified examples that we fully understood and could later use in the query system. We created twelve verified cases covering all three outputs (`unresolved`, `upward_ra`, and `downward_ra`).

We wrote `run_tests.sh` to automate execution and pass/fail checking, and `generate_json.py` to convert the test cases into `test_results.json` for later querying.

AFL++ Fuzz Testing

We also used AFL++ to generate additional inputs automatically and explore behaviors that normal test cases might miss. Since `tcas` normally takes command-line arguments, we created a separate fuzzing version of the program along with a wrapper, `tcas_wrapper.c`, that reads 12 integers from standard input and then calls `alt_sep_test()`.

We compiled the fuzzing target with `afl-clang-fast` and created 6 seed inputs. After running AFL++ for about 10 minutes, the queue expanded to 14 total inputs, 3 real crash-triggering inputs were found, and 0 hangs were discovered.

This part of the testing design was important because it showed that the program has failure cases that are not obvious from normal hand-written tests alone.

Coverage Collection with gcov

To measure the completeness of our testing, we used gcov to collect both statement and branch coverage. We compiled an instrumented version of TCAS and ran it against both the valid benchmark rows and the hand-crafted suite. This produced an aggregated profile across 1,590 total executions.

We then generated an annotated .gcov file and used parse_gcov.py to convert it into coverage_report.json. Some of the uncovered code was either logically infeasible or part of the usage-error path that the valid-input harness did not trigger.

Structured Results and Project Role

A major goal of the testing component was not just to run tests, but to store the results in a way that can later support queries. As a result, we produced structured files such as test_results.json, coverage_report.json, and afl_summary.json, along with supporting scripts and notes.

Overall, the testing component gives us the dynamic side of the project. The static analysis component explains how the code is organized, while the testing component shows what actually happens when the program runs. These results will later be connected to the storage and query parts of the project so users can ask questions about outputs, crashes, and coverage.

Program Analysis Results Storage

For our program analysis results, we decided to store them in structured JSON files. We decided to use JSON files because they can represent complex information well and are easy to parse. Additionally, it allowed us to avoid using a database, which could have made the project unnecessarily complex.

Our project stores all the JSON files in the analysis_store folder, which is contained within our query_system directory. This way, they are all stored in a centralized folder, making accessing that data simpler.

As for how the information is converted to JSON, we run multiple files to convert test results into JSON. For more information about how the data is sent to the analysis store in JSON files, see the [Visualization of the Overall Picture](#) section.

Query Interface Implementation Design

The query interface is a routing system that allows for natural language input and is built to handle queries related to our program. It processes input, extracts relevant entities, and then fetches and/or computes the query's answer from the centralized JSON analysis store. This store contains all of our results in JSON format.

When a query is submitted, the program performs the following steps:

1. Input is converted to lowercase and standardized.
2. Tokens are extracted from the input.
3. Entities are extracted from the tokens, such as variable names and function names
4. The input query is searched for keywords and scored based on what keywords are included in the prompt.
5. Then it picks the winner (highest scoring keyword category) to retrieve the answer.
6. If there is no clear match, it returns a default message stating that it cannot answer the query and directs the user to ask a different question it can answer.
7. If it finds a match, the query is routed to a specific function for its category.
8. Some error handling is done, and sometimes it asks for clarification before asking, but then it performs what it needs to find the answer.
9. Some functions simply scan the JSON to look up information, but others run algorithms such as DFS to compute the answers.
10. Finally, the answer is returned along with the information source.

Visualization of the Overall Picture

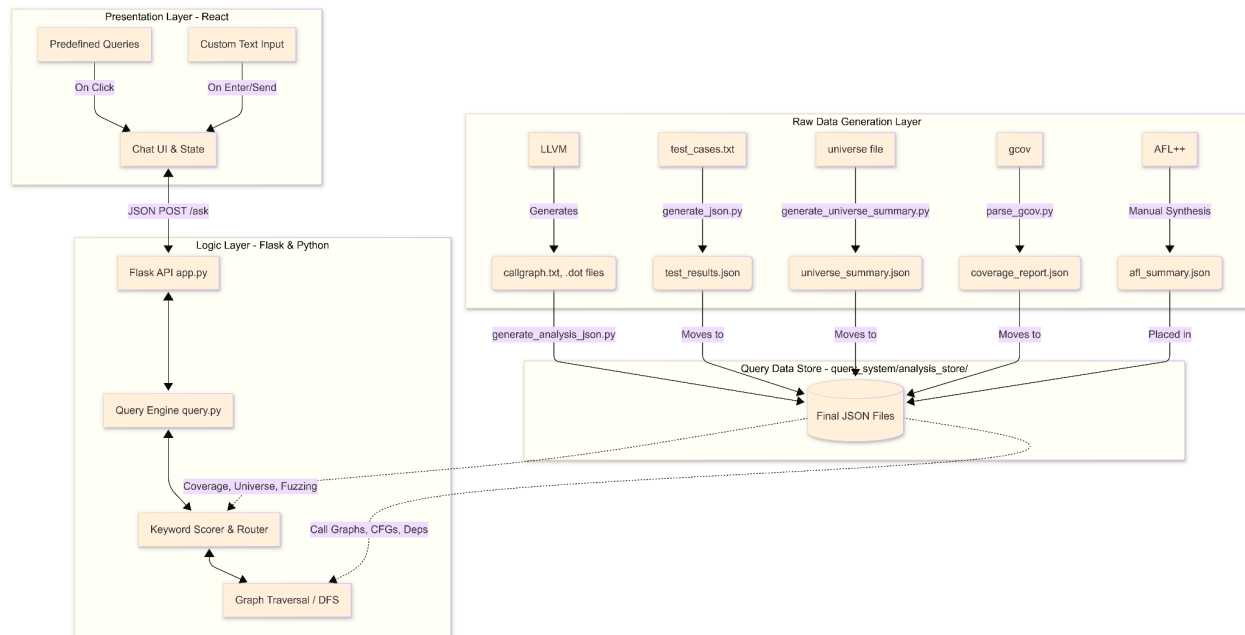


Figure 1: Architecture Diagram

The diagram above shows the architecture of our program as a whole. In the top-left corner, we have our presentation layer, which contains everything related to our user interface. We added both our custom text input (the chat) and the preset queries (on the side). When submitted, both send the data to the chat state, which then sends a request to the backend with the query. We wrapped our backend query in a Flask API in order to connect it to our React frontend. This allows users to query our analysis store directly through the chat.

Now we look at the logic layer. Then our query engine processes the query, scores it by tokens, and performs a keyword search, routing it to the most likely desired final answer. Our final answers include templates for various questions that could be asked, and we pull in data from the reads, such as variable names, numbers, etc. If the question requires any graph traversals or DFS, the program runs that function on the file to obtain the data.

Then we have the query data store, where all our JSON files contain the information from our analysis and testing. This folder is called `analysis_store`.

In the top-right corner, there is the raw data generation layer, which shows how all our information was retrieved and sent to the analysis store. We used LLVM to perform static analysis on the program, generating the .dot files in our `static_analysis` folder and information about the call graph. Then we generate JSON files from those in our `generate_analysis_json.py` file and store the structured JSON in our analysis store.

For the test cases, we created tests in `test_cases.txt`, ran them, generated a JSON file with the results, and stored it in the analysis store. We did this to understand the program and its functionality, and to test its performance on tests separate from those that came with the program.

Then, for the universe file, which is a set of tests originally created by and provided with the TCAS program, we ran the tests and generated a JSON summary of the results. This JSON was also sent to our analysis store.

To analyze our program's coverage, we used the `gcov` tool. We parsed the results in `parse_gcov.py` and then assembled another JSON file to store program-coverage results in the analysis store.

Finally, for AFL, we ran the fuzzer on our program and received a series of output files after stopping the program. Then we manually compiled the results into a `.txt` file and manually created a JSON file with the results to keep all our results in JSON format and ensure the analysis store is consistent.

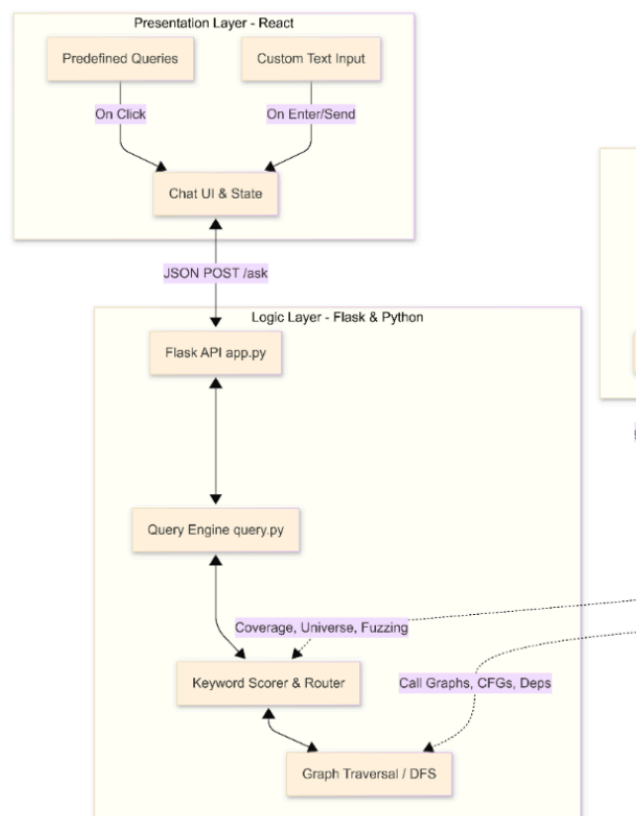


Figure 2: Left-half of the Architecture Diagram, Presentation and Logic Layer

This is a zoomed-in view of the presentation and logic layers of our program's architecture.

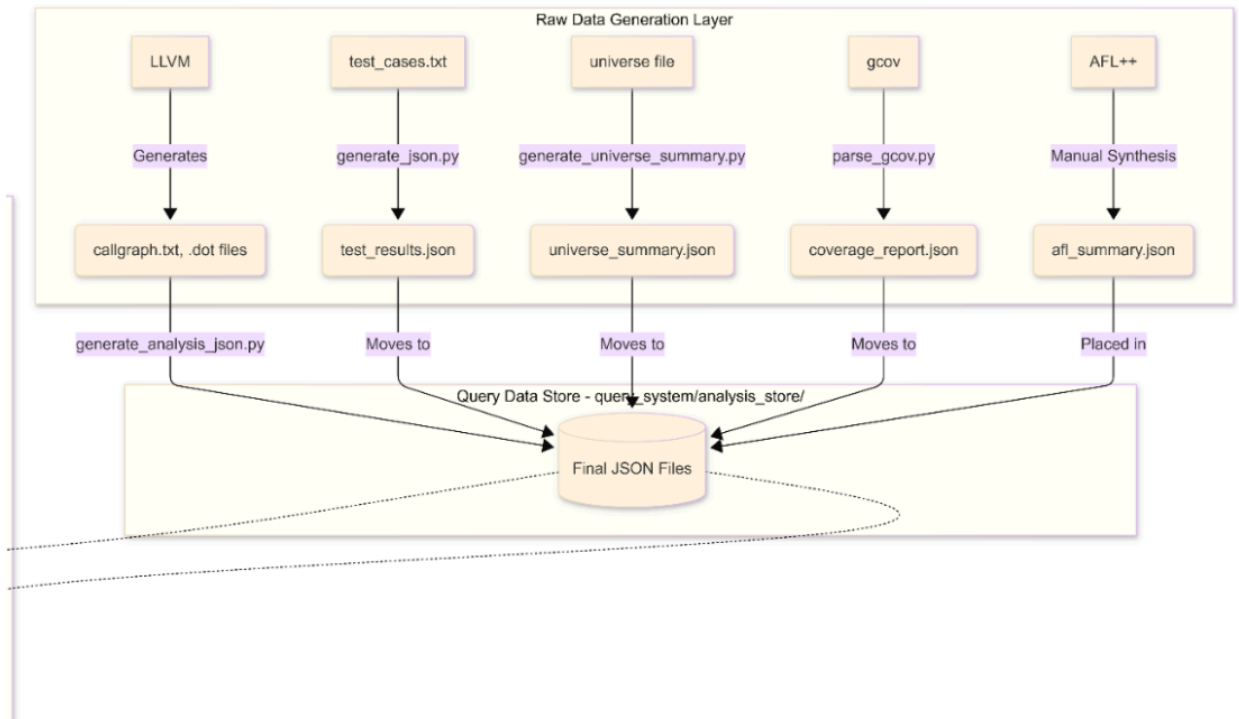


Figure 3: Right-half of Architecture Diagram, Raw Data Generation Layer

This is a zoomed-in view of the raw data generation layer and the query data store in our architecture diagram.

UI and Final Product

For our final product, we designed an interactive website.

We built the user interface using React, which leverages components, state management, and event handling to create an app that lets users navigate between views. These views include the home page (where users can run queries), description and images of the control flow graph and interprocedural call graph, as well as informational pages about our team and the TCAs program. Users run queries from the home page, where they can either select from predefined questions or input their own custom queries! These queries are sent to our backend, which was created in Project Update 3, which processes the request and returns analysis results.

Not sure what to ask?

Choose from our list of predefined questions!

CALL GRAPH >

CONTROL FLOW GRAPH >

DEPENDENCIES >

COVERAGE >

TESTING >

FUZZING >

Select a question from the left or type your own to start analyzing TCAS!

Type a custom query here...



Figure 4: User Interface Home Page

This is an image of our home page, including all those features mentioned such as the conversational chat, the preset questions on the side, along with all of our buttons and tabs. Where it says “CS 4130” it links to the home page.

Control Flow Graph

The control flow graphs (CFGs) below visualize the flow of execution through the TCAS logic, with each node representing a basic block of instructions and edges representing possible execution paths.

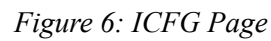
Analyzing the CFGs allowed us to understand the program's branch structure and trace how logic flows through its various conditions and return paths!



Figure 5: CFG Page

We also have buttons that display the CFGs for the TCAS program. In the picture above, we show the beginning of the CFG page that includes a short description and the start of a CFG diagram. There are more CFG diagrams displayed on the site, but we did not take images of each of them for this report.

The Interprocedural Control Flow Graph (ICFG) extends the CFG by connecting the control flow across all functions! The ICFG captures how execution transfers between call sites and their callees. This allows us to reason about the program's behavior holistically, revealing cross-function dependencies and execution paths that wouldn't be visible when analyzing functions by themselves.



This is the page that displays the ICFG for the TCAS program.

About Us

We are a team of 5 undergraduate students, attending Iowa State University, who are enrolled in COMS 4130 for the Spring 2026 semester!



Kennedy Wendl is a senior majoring in Computer Science and Data Science! Outside of class, she loves to hangout with friends or go for a run. Her favorite lesson from CS 4130 was learning how to draw a CFG.



Figure 7: About Us Page

Above is a picture of the start of our About Us page. It includes information about all of our teammates, along with our favorite parts of COMS 4130.

What is TCAS?

The Traffic Alert and Collision Avoidance System (TCAS) is an aircraft collision avoidance system designed to reduce the incidence of mid-air collisions between aircraft. It monitors the airspace around an aircraft for other aircraft equipped with a corresponding active transponder, independent of air traffic control.

When a potential threat is detected, TCAS provides the pilots with visual and audible advisories. These include Traffic Advisories (TAs), which alert the crew to a nearby aircraft, and Resolution Advisories (RAs), which command the crew to maneuver the aircraft (e.g., "Climb", "Descend") to maintain safe separation.

This static analysis project explores a simplified version of the TCAS resolution logic, analyzing its control flow, data dependencies, and coverage characteristics.

Figure 8: About TCAS Page

We also created an about TCAS page for users to get a quick understanding of what the TCAS program does. As you can see in the photo above as well, we have a GitHub tab that links to our project's Github.

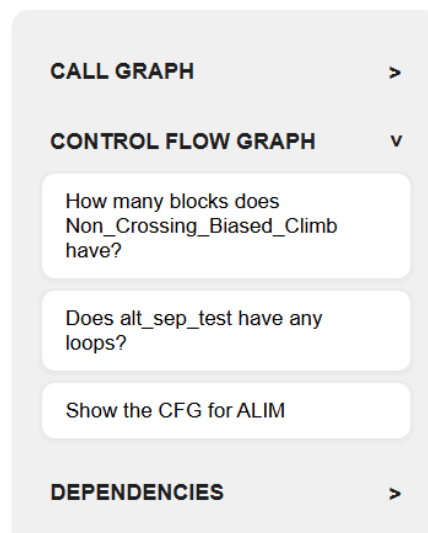


Figure 9: Preset Query Dropdown

We also have our preset questions in a side bar dropdown menu which is shown in the image above.

Exceptional Features

For our exceptional features, our group went above and beyond. We created many more queries than the required three, and our application contains 17 preset questions that users can ask. Not only does it support those 17 queries, but it can also accept natural-language queries that are similar but not identical. We are able to do this because our query system was designed this way.

We also created an entire user interface, enhancing the appeal of our application and making it easy to use and understand. It is highly reactive and can adapt to a wide range of screen sizes. Implementing a chat, it makes the user feel like they are having a conversation with the application about their questions. We also display the preset questions, ready to ask without requiring the user to type.

We also added an “About Us” page to provide information about our team and an “About TCAS” page to inform users about the primary purpose of the original program that our application addresses.

Along with that, we also added visual diagrams of our CFGs and ICFGs to the website in the upper-right corner, with buttons. We wanted to add this visual to the website in case users want to view it themselves, rather than asking questions about it in the chat.